МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 2

Тема: **Порождающие паттерны: Абстрактная фабрика, Фабричный метод, Прототип**

Дисциплина: **«Паттерны проектирования»**

**Выполнила:**

Студентка группы ПИЖ-б-о-23-2
направление подготовки 09.03.04
«Программная инженерия»

Силина Оксана Романовна

**Проверил:**

Ассистент ДЦРСИЭ

Белик А. И.

Ставрополь 2026

**Цель работы:** изучить принцип работы порождающих паттернов проектирования «Фабричный метод» и «Прототип» и реализовать прикладную задачу расчёта траектории движения тела с использованием данных паттернов.

**Задача:** разработать приложение для расчёта параметров движения тела, брошенного под углом к горизонту, с использованием двух порождающих паттернов проектирования.

Выбранные паттерны

- Фабричный метод — для создания объектов различных типов движений;
- Прототип — для клонирования готовых конфигураций расчёта.

**Индивидуальный вариант:** 3. Разработка приложения «Траектория движения»

**Репозиторий:** https://github.com/tsukuri1/patterns

## 1. Описание паттернов

### 1.1. Фабричный метод (Factory Method)

Фабричный метод — это порождающий паттерн проектирования, который определяет интерфейс для создания объекта, но позволяет подклассам изменять тип создаваемого объекта. Паттерн делегирует создание объектов наследникам.

Применение в задаче:

Создаётся абстрактный класс MotionFactory с методом create_motion(). Конкретные фабрики (IdealMotionFactory, DragMotionFactory) создают соответствующие объекты движений:

- IdealMotion — движение без сопротивления воздуха;
- DragMotion — движение с учётом сопротивления воздуха.

### 1.2. Прототип (Prototype)

Прототип — это порождающий паттерн проектирования, который позволяет копировать объекты, не вдаваясь в подробности их реализации. Вместо создания нового объекта «с нуля» мы клонируем уже существующий.

Применение в задаче:

Класс MotionConfig хранит параметры расчёта (начальная скорость, угол, шаг интегрирования и т.д.) и реализует метод clone(), позволяющий быстро создавать новые конфигурации на основе уже существующих.

## 2. Структура приложения

Приложение состоит из следующих компонентов:

| Компонент | Назначение |
|---|---|
| Motion | Абстрактный класс движения (интерфейс) |
| IdealMotion | Движение без сопротивления воздуха |
| DragMotion | Движение с сопротивлением воздуха |
| MotionFactory | Абстрактная фабрика движений |
| IdealMotionFactory | Фабрика идеального движения |
| DragMotionFactory | Фабрика движения с сопротивлением |
| MotionConfig | Конфигурация расчёта (Прототип) |
| TrajectoryApp | Графический интерфейс приложения |

## 3. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

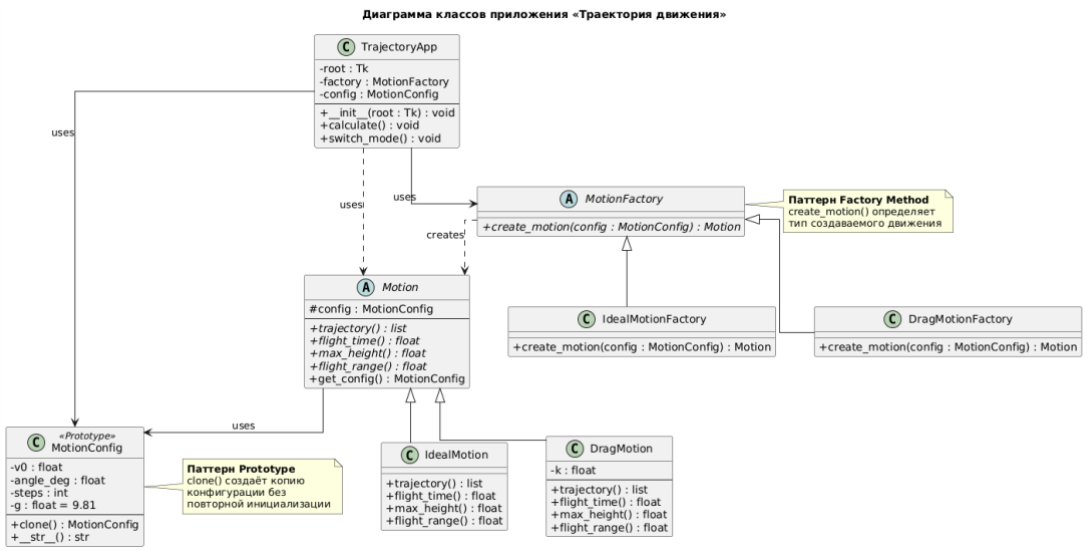



Рисунок 1 — Диаграмма классов приложения «Траектория движения»

Диаграмма содержит:

- Абстрактный класс Motion и его наследников IdealMotion, DragMotion;
- Абстрактную фабрику MotionFactory и конкретные фабрики;
- Класс MotionConfig, реализующий паттерн «Прототип»;
- Класс TrajectoryApp — интерфейс приложения.

## 4. Реализация паттернов

### 4.1. Паттерн «Прототип»

Метод clone() реализован через copy.deepcopy(), что гарантирует полную независимость копии от оригинала: изменение параметров копии не повлияет на исходную конфигурацию.

```
import copy

class MotionConfig:
    """Конфигурация расчёта. Реализует паттерн Прототип."""
    def __init__(self, v0=20.0, angle_deg=45.0, steps=50, g=9.81):
        self.v0 = v0
        self.angle_deg = angle_deg
        self.steps = steps
```

```
        self.g = g

    def clone(self):
        """Создаёт копию объекта (Prototype)."""
        return copy.deepcopy(self)

    def __str__(self):
        return (f"MotionConfig(v0={self.v0}, angle={self.angle_deg}, "
                f"steps={self.steps}, g={self.g})")
```

### 4.2 Паттерн «Фабричный метод»

Абстрактный класс Motion задаёт общий интерфейс для всех типов движения, а конкретные реализации переопределяют методы расчёта. Фабрики инкапсулируют создание объектов, благодаря чему клиентский код не зависит от конкретных классов движений.

```
import math

class Motion:
    """Абстрактный класс движения."""
    def __init__(self, config: MotionConfig):
        self.config = config

    def flight_time(self):
        raise NotImplementedError

    def max_height(self):
        raise NotImplementedError

    def flight_range(self):
        raise NotImplementedError

    def trajectory(self):
        raise NotImplementedError

    def get_config(self):
        return self.config
```

```
class IdealMotion(Motion):
    """Движение без сопротивления воздуха."""
    def flight_time(self):
        a = math.radians(self.config.angle_deg)
        return 2 * self.config.v0 * math.sin(a) / self.config.g

    def max_height(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * (math.sin(a) ** 2) / (2 * self.config.g)

    def flight_range(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * math.sin(2 * a) / self.config.g

    def trajectory(self):
        T = self.flight_time()
        pts = []
        a = math.radians(self.config.angle_deg)
        for i in range(self.config.steps + 1):
            t = T * i / self.config.steps
            x = self.config.v0 * math.cos(a) * t
            y = self.config.v0 * math.sin(a) * t - 0.5 * self.config.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class DragMotion(Motion):
    """Движение с учётом сопротивления воздуха (упрощённая модель)."""
    def __init__(self, config: MotionConfig, k=0.1):
        super().__init__(config)
        self.k = k

    def flight_time(self):
        return 2 * self.config.v0 * math.sin(math.radians(self.config.angle_deg)) \
               / (self.config.g * (1 + self.k))
```

```
    def max_height(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * (math.sin(a) ** 2) \
               / (2 * self.config.g * (1 + self.k))

    def flight_range(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * math.sin(2 * a) \
               / (self.config.g * (1 + self.k))

    def trajectory(self):
        T = self.flight_time()
        pts = []
        a = math.radians(self.config.angle_deg)
        for i in range(self.config.steps + 1):
            t = T * i / self.config.steps
            x = self.config.v0 * math.cos(a) * t / (1 + self.k * t)
            y = self.config.v0 * math.sin(a) * t - 0.5 * self.config.g * t ** 2
            pts.append((round(x, 3), round(max(y, 0), 3)))
        return pts


class MotionFactory:
    """Абстрактная фабрика движений."""
    def create_motion(self, config: MotionConfig) -> Motion:
        raise NotImplementedError


class IdealMotionFactory(MotionFactory):
    def create_motion(self, config: MotionConfig) -> Motion:
        return IdealMotion(config)


class DragMotionFactory(MotionFactory):
    def create_motion(self, config: MotionConfig) -> Motion:
        return DragMotion(config, k=0.1)
```

### 4.3. Графический интерфейс

Графический интерфейс выполняет роль витрины, через которую видно работу обоих паттернов. Переключатель режимов «Без сопротивления/С сопротивлением» наглядно демонстрирует, как фабричный метод выбирает нужную фабрику и создаёт соответствующий объект движения, а кнопка «Клонировать конфигурацию» позволяет в реальном времени убедиться, что прототип создаёт независимую копию. Кроме того, интерфейс обеспечивает удобство использования приложения: пользователь вводит параметры, получает результаты расчёта и график траектории.

```
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt


class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        self.config = MotionConfig()
        self.factory = IdealMotionFactory()

        root.title("Траектория движения")

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0, column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0, sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Режим:").grid(row=2, column=0, sticky="w")
```

```
        self.mode_var = tk.StringVar(value="ideal")
        tk.Radiobutton(root, text="Без сопротивления",
                       variable=self.mode_var, value="ideal",
                       command=self.switch_mode).grid(row=2, column=1,
sticky="w")
        tk.Radiobutton(root, text="С сопротивлением",
                       variable=self.mode_var, value="drag",
                       command=self.switch_mode).grid(row=3, column=1,
sticky="w")

        tk.Button(root, text="Рассчитать",
                  command=self.calculate).grid(row=4, column=0, columnspan=2,
pady=5)
        tk.Button(root, text="Клонировать конфигурацию",
                  command=self.clone_config).grid(row=5, column=0,
columnspan=2)

        self.result = tk.Text(root, width=45, height=10)
        self.result.grid(row=6, column=0, columnspan=2)

    def switch_mode(self):
        """Переключение фабрики в зависимости от режима."""
        if self.mode_var.get() == "ideal":
            self.factory = IdealMotionFactory()
        else:
            self.factory = DragMotionFactory()

    def clone_config(self):
        """Демонстрация паттерна Прототип."""
        self.config = self.config.clone()
        messagebox.showinfo("Прототип",
                            f"Создана копия конфигурации:\n{self.config}")

    def calculate(self):
        try:
            self.config.v0 = float(self.v0_entry.get())
            self.config.angle_deg = float(self.angle_entry.get())
            if self.config.v0 <= 0:
                raise ValueError("Скорость должна быть > 0")
```

```
            if not (0 < self.config.angle_deg < 90):
                raise ValueError("Угол должен быть в диапазоне (0; 90)")

            motion = self.factory.create_motion(self.config)
            T = motion.flight_time()
            H = motion.max_height()
            L = motion.flight_range()
            pts = motion.trajectory()

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Тип движения: {type(motion).__name__}\n"
                f"Время полёта: {T:.3f} c\n"
                f"Макс. высота: {H:.3f} м\n"
                f"Дальность: {L:.3f} м\n")

            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title(f"{type(motion).__name__}: v0={self.config.v0}, "
                      f"угол={self.config.angle_deg}°")
            plt.xlabel("X, м")
            plt.ylabel("Y, м")
            plt.grid(True)
            plt.axis('equal')
            plt.show()
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
    root.mainloop()
```

## 5. Результаты работы программы

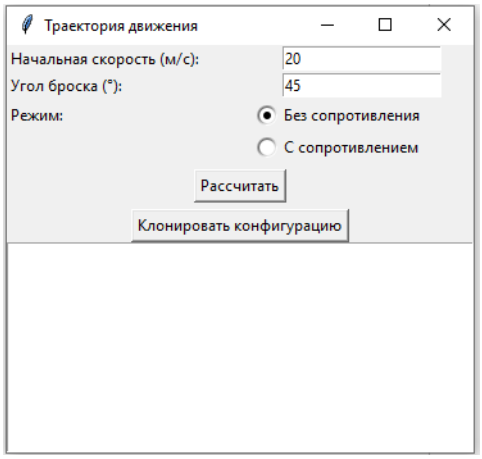



Рисунок 2 — Главное окно приложения

Пользователь вводит начальную скорость, угол броска и выбирает режим:

- «Без сопротивления» — используется IdealMotionFactory;
- «С сопротивлением» — используется DragMotionFactory.

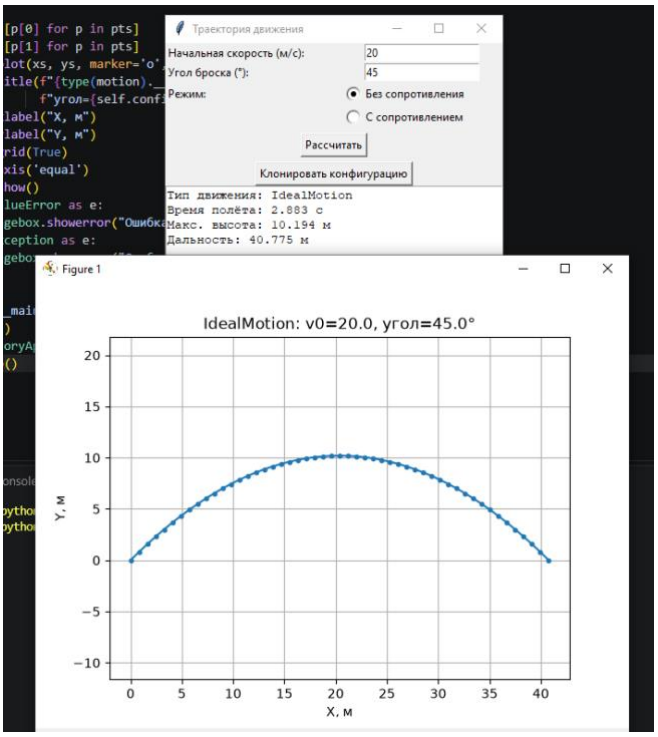



Рисунок 3 — Расчёт идеального движения

При v0 = 20 м/с, угол = 45°:

Время полёта: 2.883 с

Макс. высота: 10.194 м

Дальность: 40.774 м

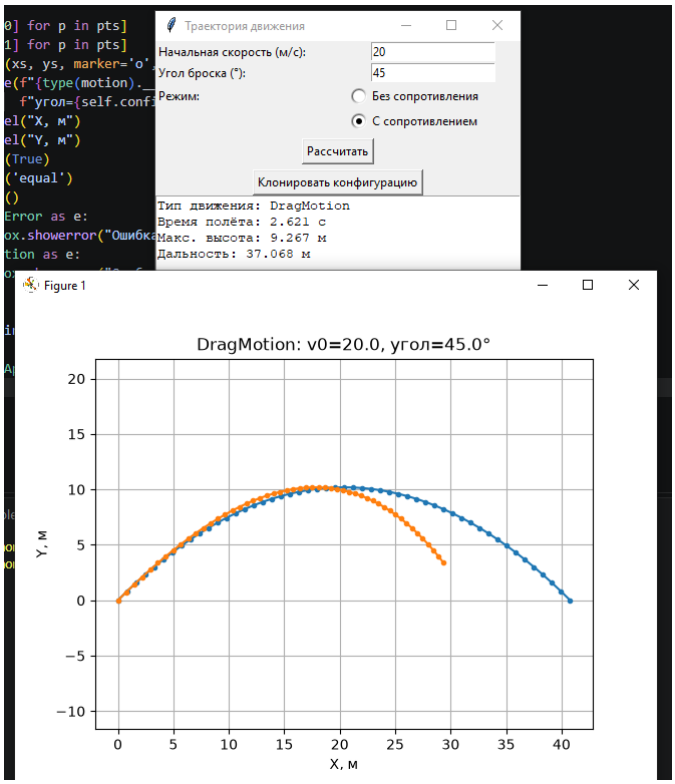



Рисунок 4 — Расчёт движения с сопротивлением

При тех же параметрах дальность и высота меньше за счёт сопротивления.

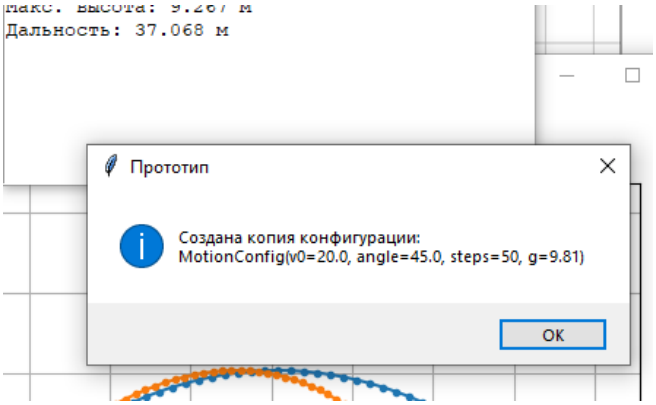



Рисунок 5 — Демонстрация паттерна Прототип

Кнопка «Клонировать конфигурацию» создаёт независимую копию текущей конфигурации. Изменение оригинала не влияет на копию.

**6. Исходный код**

```
import math
import copy
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt


class MotionConfig:
    """Конфигурация расчёта. Реализует паттерн Прототип."""
    def __init__(self, v0=20.0, angle_deg=45.0, steps=50, g=9.81):
        self.v0 = v0
        self.angle_deg = angle_deg
        self.steps = steps
        self.g = g

    def clone(self):
        return copy.deepcopy(self)

    def __str__(self):
        return (f"MotionConfig(v0={self.v0}, angle={self.angle_deg}, "
                f"steps={self.steps}, g={self.g})")


class Motion:
    """Абстрактный класс движения."""
    def __init__(self, config: MotionConfig):
        self.config = config

    def flight_time(self): raise NotImplementedError
    def max_height(self): raise NotImplementedError
    def flight_range(self): raise NotImplementedError
    def trajectory(self): raise NotImplementedError
    def get_config(self): return self.config
```

```
class IdealMotion(Motion):
    def flight_time(self):
        a = math.radians(self.config.angle_deg)
        return 2 * self.config.v0 * math.sin(a) / self.config.g

    def max_height(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * (math.sin(a) ** 2) / (2 * self.config.g)

    def flight_range(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * math.sin(2 * a) / self.config.g

    def trajectory(self):
        T = self.flight_time()
        pts = []
        a = math.radians(self.config.angle_deg)
        for i in range(self.config.steps + 1):
            t = T * i / self.config.steps
            x = self.config.v0 * math.cos(a) * t
            y = self.config.v0 * math.sin(a) * t - 0.5 * self.config.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class DragMotion(Motion):
    def __init__(self, config: MotionConfig, k=0.1):
        super().__init__(config)
        self.k = k

    def flight_time(self):
        return 2 * self.config.v0 * math.sin(math.radians(self.config.angle_deg))
\
               / (self.config.g * (1 + self.k))

    def max_height(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * (math.sin(a) ** 2) \
               / (2 * self.config.g * (1 + self.k))
```

```
    def flight_range(self):
        a = math.radians(self.config.angle_deg)
        return (self.config.v0 ** 2) * math.sin(2 * a) \
               / (self.config.g * (1 + self.k))

    def trajectory(self):
        T = self.flight_time()
        pts = []
        a = math.radians(self.config.angle_deg)
        for i in range(self.config.steps + 1):
            t = T * i / self.config.steps
            x = self.config.v0 * math.cos(a) * t / (1 + self.k * t)
            y = self.config.v0 * math.sin(a) * t - 0.5 * self.config.g * t ** 2
            pts.append((round(x, 3), round(max(y, 0), 3)))
        return pts


class MotionFactory:
    def create_motion(self, config: MotionConfig) -> Motion:
        raise NotImplementedError


class IdealMotionFactory(MotionFactory):
    def create_motion(self, config: MotionConfig) -> Motion:
        return IdealMotion(config)


class DragMotionFactory(MotionFactory):
    def create_motion(self, config: MotionConfig) -> Motion:
        return DragMotion(config, k=0.1)


class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        self.config = MotionConfig()
        self.factory = IdealMotionFactory()
```

```
        root.title("Траектория движения")

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Режим:").grid(row=2, column=0, sticky="w")
        self.mode_var = tk.StringVar(value="ideal")
        tk.Radiobutton(root, text="Без сопротивления",
                       variable=self.mode_var, value="ideal",
                       command=self.switch_mode).grid(row=2,
column=1, sticky="w")
        tk.Radiobutton(root, text="С сопротивлением",
                       variable=self.mode_var, value="drag",
                       command=self.switch_mode).grid(row=3,
column=1, sticky="w")

        tk.Button(root, text="Рассчитать",
                  command=self.calculate).grid(row=4, column=0, columnspan=2,
pady=5)
        tk.Button(root, text="Клонировать конфигурацию",
                  command=self.clone_config).grid(row=5,
column=0, columnspan=2)

        self.result = tk.Text(root, width=45, height=10)
        self.result.grid(row=6, column=0, columnspan=2)

    def switch_mode(self):
        if self.mode_var.get() == "ideal":
            self.factory = IdealMotionFactory()
        else:
```

```
        self.factory = DragMotionFactory()

    def clone_config(self):
        self.config = self.config.clone()
        messagebox.showinfo("Прототип",
                            f"Создана копия конфигурации:\n{self.config}")

    def calculate(self):
        try:
            self.config.v0 = float(self.v0_entry.get())
            self.config.angle_deg = float(self.angle_entry.get())
            if self.config.v0 <= 0:
                raise ValueError("Скорость должна быть > 0")
            if not (0 < self.config.angle_deg < 90):
                raise ValueError("Угол должен быть в диапазоне (0; 90)")

            motion = self.factory.create_motion(self.config)
            T = motion.flight_time()
            H = motion.max_height()
            L = motion.flight_range()
            pts = motion.trajectory()

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Тип движения: {type(motion).__name__}\n"
                f"Время полёта: {T:.3f} c\n"
                f"Макс. высота: {H:.3f} м\n"
                f"Дальность: {L:.3f} м\n")

            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title(f"{type(motion).__name__}: v0={self.config.v0}, "
                      f"угол={self.config.angle_deg}°")
            plt.xlabel("X, м")
            plt.ylabel("Y, м")
            plt.grid(True)
            plt.axis('equal')
            plt.show()
```

```
            except ValueError as e:
                messagebox.showerror("Ошибка ввода", str(e))
            except Exception as e:
                messagebox.showerror("Ошибка", str(e))


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
    root.mainloop()
```

**Вывод:**

В ходе выполнения лабораторной работы было разработано приложение «Траектория движения», реализующее расчёт параметров движения тела, брошенного под углом к горизонту.

В приложении применены два порождающих паттерна проектирования:

1. Фабричный метод — для создания объектов различных типов движения (IdealMotion, DragMotion) через соответствующие фабрики (IdealMotionFactory, DragMotionFactory). Это позволяет легко расширять приложение новыми типами движений.

2. Прототип — для клонирования конфигураций расчёта (MotionConfig). Метод clone() позволяет создавать независимые копии параметров без повторной инициализации.

Таким образом, была изучена практическая реализация двух порождающих паттернов и получено работоспособное приложение с графическим интерфейсом для расчёта и визуализации траектории движения тела.